

Intelligent tuning of PID controllers: Comprehensive approach based on modified Particle Swarm Optimization (PSO) algorithm.

1st Julio César Zambrano Abad

Research Group in Interaction, Robotics, and Automation
Salesian Polytechnic University
Cuenca, Ecuador
<https://orcid.org/0000-0002-2935-8458>

2nd Marco Alexander Carpio Alemán

Research Group in Interaction, Robotics, and Automation
Salesian Polytechnic University
Cuenca, Ecuador
<https://orcid.org/0000-0001-9666-1775>

Abstract—This paper introduces a novel approach for tuning PID controllers by framing the tuning problem as an optimization task, solved using a modified version of Particle Swarm Optimization (PSO). The modified PSO algorithm achieves high precision without requiring manual adjustment of hyper-parameters. Unlike conventional optimization-based tuning methods that focus solely on minimizing a specific type of error, this approach employs a multi-objective formulation to simultaneously minimize Integral Absolute Error (IAE), control action aggressiveness, and settling time. The optimization problem is treated as a single-objective problem with weighted indices for each minimization goal. To enhance adaptability in controller tuning, the optimization problem incorporates constraints to precisely manage specific parameters of both the system response and the control action. Users can adjust the constraints and weights to achieve precise and adaptable control over the system's overshoot and settling time. Additionally, the amplitude and maximum variation of the control signal can be regulated, resulting in a versatile solution adaptable to various design requirements. This approach was evaluated on a thermal process model based on a Peltier cell. The results underscore the effectiveness of this method, offering a comprehensive and versatile tool for the precise tuning of PID controllers.

Index Terms—Particle Swarm Optimization, PID controllers, multi-objective.

I. INTRODUCTION

The Proportional-Integral-Derivative (PID) control is one of the most widely used control techniques in the industry due to its ease of implementation and reliability [1], [2]. The optimal performance of a PID controller is closely linked to the precise adjustment of its tuning parameters: the proportional gain (K_p), integral time (T_i), and derivative time (T_d). These parameters influence the controller's ability to maintain stability, minimize steady-state error, and effectively respond to system disturbances.

Currently, the literature offers many methods for tuning PID controllers [3]. One of the most popular is the Ziegler-Nichols method, introduced in the early 1940s and still widely recognized and utilized today [4], [5]. In addition to Ziegler-Nichols, there are other empirical tuning methods such as the Cohen-Coon Method [6], the Internal Model Control (IMC) method [7], among others [8]. Empirical methods are popular

due to their ability to achieve satisfactory tuning quickly, often without requiring a process model or specialized software. However, there is likely a set of parameters that provides better tuning for the controller.

Intelligent methods enhance PID controller tuning by using optimization algorithms and computational intelligence to efficiently automate the search for precise solutions. In this context, bio-inspired algorithms have been widely used. Currently, applications of PID controller tuning using genetic algorithms [9], [10], differential evolution [11], [12], and PSO [13]–[15] are prevalent. Other intelligent methods have also been explored for optimal PID tuning [16].

Intelligent methods address PID controller tuning by optimizing to minimize error metrics like the Integral of Time multiplied by Absolute Error (ITAE), Integral of Absolute Error (IAE), or Integral of Squared Error (ISE). While error minimization is crucial for PID controller performance, other factors such as output signal overshoot, maximum control signal amplitude, and controller aggressiveness should also be considered. Minimizing error alone may drive the control action to reach the reference quickly; however, this can lead to exceeding the allowable operating limits. On the other hand, rapid changes in control action within set time frames aren't always feasible; for example, a valve can't instantly open from 0 to 100%. Conversely, excessive overshoot in the system output is also a common issue.

To enhance system control and regulate the control action effectively, additional objectives must be integrated into the optimization problem. While incorporating multiple objectives into PID controller tuning is established [17], this article distinguishes itself by highlighting the significance of control signal increments. This perspective adds a new dimension to analyzing controller performance, as increments can directly impact the aggressiveness and stability of the system. Additionally, while other proposals typically focus solely on presenting output results and neglect the behavior of the control action, this approach comprehensively addresses both the output and the control action, providing a more complete and accurate assessment of controller performance.

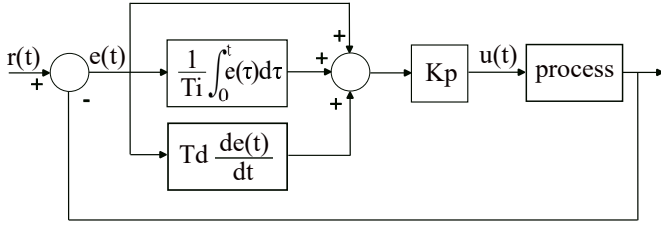


Fig. 1. Basic control loop with PID controller

Tuning PID controllers, particularly when considering multiple optimization objectives and constraints, is inherently complex and not feasibly solvable using traditional optimization methods. PSO emerges as an ideal approach to address these issues, thanks to its ability to adapt and optimize based on bio-inspired principles [18]. PSO is highly regarded for its low computational cost, ease of implementation, and notable accuracy. Over time, PSO has evolved significantly, resulting in various variants that enhance its precision and convergence speed [19]. These advancements have made PSO a robust and versatile optimization method, effectively addressing complex problems with multiple constraints and objectives.

From now on, this article is organized as follows: Section II provides a concise review of PID controllers and PSO, highlighting key aspects of both topics. Section III presents the optimization problem, detailing the objectives to be minimized and the constraints considered. Section IV presents the results and discussion. Finally, some conclusions are reported in Section V.

II. BACKGROUND

Before detailing the proposed tuning approach, preliminary concepts about PID controllers and PSO, including a modified version of the PSO algorithm, are provided.

A. PID controllers

The PID controller uses information from the closed-loop error $e(t)$, calculated as the difference between the measured output $y(t)$ and the set-point or desired value $r(t)$ (see Fig. 1). The control action $u(t)$ is adjusted by the controller to achieve satisfactory closed-loop performance, based on the error, by combining the action of three terms: proportional, integral, and derivative.

The classic form of a parallel PID controller in continuous time is expressed as follows:

$$u(t) = K_p \left[e(t) + \frac{1}{T_i} \cdot \int_0^t e(\tau) d\tau + T_d \cdot \frac{d}{dt} e(t) \right] \quad (1)$$

where:

- $K_p \in \mathbb{R}$ is the proportional gain, which adjusts the control response proportionally to the current error. This parameter mainly influences the system's response speed.
- $T_i \in \mathbb{R}$ is the integral time, which determines the controller's response to the accumulation of errors over

time. This parameter helps to correct the error in steady-state.

- $T_d \in \mathbb{R}$ is the derivative time, which determines the controller's response to rapid changes in error. This parameter adds damping to the system, preventing excessive oscillations.

Tuning a PID controller involves adjusting the parameters K_p , T_i , and T_d to achieve optimal performance of the controlled system.

B. PSO

PSO is a stochastic algorithm inspired by the social behavior of flocks of birds and schools of fish. It uses a population of particles, each with a position and velocity in the search space. The position represents a potential solution, while the velocity directs the particle's movement, analogous to how birds navigate their environment [20]. The algorithm operates by iteratively updating the positions and velocities of swarm particles as they search for the global minimum within the solution space:

$$v_i^{t+1} = \omega v_i^t + c_1 \cdot r_1 \cdot (p_i^t - x_i^t) + c_2 \cdot r_2 \cdot (p_g^t - x_i^t) \quad (2)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (3)$$

where:

- i denotes the index of the particle, and t represents the current iteration number.
- x and v represent the position and velocity of each particle, respectively.
- ω is the inertia weight that controls the influence of a particle's previous velocity on its current one.
- p contains the particle's best historical position, resembling how a bird remembers its best location to find food.
- p_g represents the best global position or the best solution found by any particle in the swarm. This concept mirrors how birds share information about food locations to enhance the swarm's overall search performance.
- c_1 is known as the cognitive learning factor.
- c_2 is known as the social coefficient.
- r_1 and r_2 are uniformly distributed random numbers with range (0,1) that serve to diversify the search, preventing particles from prematurely converging towards local regions of the search space.

The equation for updating velocity in PSO has three terms. The inertia term, controlled by the tuning parameter ω , influences the particle's current velocity based on its previous velocity. A low ω value promotes exploration, while a high ω value favors exploitation by maintaining the particle's direction. The second term is known as the cognitive component. This term considers each particle's best experience to guide its movement. The third term is known as the social component, referring to the influence of the best-known point by the swarm as a whole. The parameters ω , c_1 , and c_2 regulate the influence of the three terms on the particle's new velocity and are considered tuning parameters of the algorithm.

The PSO algorithm (see Algorithm 1) starts by initializing a population of particles with random positions and velocities. Given that there is no existing record of the best positions for the particles, the best local positions are initialized with the particles' initial positions ($p_i^0 = x_i^0$). Then, the objective function is evaluated for each particle and the best global position is initialized (p_g^0). In each iteration, the particles' velocities and positions are updated, applying range corrections if needed. The iteration ends by updating the personal and global best positions.

Algorithm 1 Particle Swarm Optimization (PSO)

- 1: Define swarm size (N)
 - 2: Define ω , c_1 , and c_2
 - 3: Initialize particles' positions and velocities
 - 4: Initialize personal best position (p_i^0)
 - 5: Initialize global best position (p_g^0)
 - 6: **while** not termination condition met **do**
 - 7: **for** each particle i **do**
 - 8: Update velocity using (2)
 - 9: Update position using (3)
 - 10: Correct position if necessary
 - 11: Update personal best position
 - 12: Update global best position
 - 13: **end for**
 - 14: **end while**
-

C. Modified PSO algorithm

Tuning the hyper-parameters ω , c_1 , and c_2 in the PSO algorithm is challenging, as optimal values can vary between optimization problems. It has been shown that dynamic adjustment of these parameters during iterations is more effective. The modification of the PSO algorithm for this proposal is based on the linear and automatic variation of the parameters ω , c_1 , and c_2 . To achieve this, the linear decrement of the inertia weight (LDIW) strategy presented by [21] is combined with the time-varying acceleration coefficients strategy proposed by [22]. The latter strategy refers to the linear variation of the cognitive learning factor and the social coefficient. The variation of the three parameters can be mathematically represented as follows:

$$\omega(t) = \omega_{ini} + \frac{t_{max} - t}{t_{max}} (\omega_{ini} - \omega_{end}) \quad (4)$$

$$c_1(t) = c_{1ini} + \frac{t_{max} - t}{t_{max}} (c_{1ini} - c_{1end}) \quad (5)$$

$$c_2(t) = c_{2ini} + \frac{t_{max} - t}{t_{max}} (c_{2ini} - c_{2end}) \quad (6)$$

where t_{max} is the maximum number of iterations of the algorithm. The parameter $\omega(t)$ decreases from ω_{ini} to ω_{end} with recommended values of 0.9 and 0.4, respectively. On the other hand, c_{1ini} and c_{1end} are the initial and final values of $c_1(t)$, recommended to be set at 2.5 and 0.5, respectively. Similarly, c_{2ini} and c_{2end} are the initial and final values of $c_2(t)$, recommended to be set at 0.5 and 2.5, respectively.

Specifically, the outlined modifications require minor changes to the original PSO algorithm. Referencing Algorithm 1, line 2 should be removed since tuning the three parameters is no longer necessary. Additionally, the computation of the parameters $\omega(t)$, $c_1(t)$, and $c_2(t)$ should be added before line 7 to ensure they are updated in each iteration t .

III. PROBLEM STATEMENT

The optimization problem aims to minimize the objective function $J(\theta)$, which is defined as:

$$\min_{\theta} J(\theta) = \alpha \cdot J_1(\theta) + \beta \cdot J_2(\theta) + \gamma \cdot J_3(\theta) \quad (7)$$

subject to:

$$\frac{\max(|y(k)|) - |y_{ss}|}{|y_{ss}|} * 100\% \leq Mp_r, \quad k = [1 \dots k_n]$$

$$U_{min} \leq u(k) \leq U_{max}, \quad k = [1 \dots k_n]$$

$$\max(|u(k) - u(k-1)|) \leq \Delta u_{max}, \quad k = [2 \dots k_n]$$

$$K_{pmin} \leq K_p \leq K_{pmax},$$

$$T_{imin} \leq T_i \leq T_{imax},$$

$$T_{dmin} \leq T_d \leq T_{dmax}.$$

where $\theta = [K_p, T_i, T_d]$ represents the vector of PID controller parameters, and α , β , and γ are weighting coefficients. $J_1(\theta)$ denotes the IAE (8), $J_2(\theta)$ denotes the accumulation of changes in the control action (9), and $J_3(\theta)$ denotes the settling time using the 2% criterion:

$$J_1(\theta) = \sum_{k=k_s}^{k_n} |e(k)| \quad (8)$$

$$J_2(\theta) = \sum_{k=2}^{k_n} |u(k) - u(k-1)| \quad (9)$$

$$J_3(\theta) = t : |y(k) - y_{ss}| \leq 0.02 \cdot |y_{ss}|, \quad k = [1 \dots k_n] \quad (10)$$

The parameter K_n represents the total number of samples of the corresponding signals used for optimization, while y_{ss} is the steady-state value of the system output. The first constraint limits the overshoot of $y(k)$ with M_{p_r} representing the maximum allowable. The second constraint restricts the magnitude of $u(k)$ within the bounds defined by a minimum (U_{min}) and a maximum (U_{max}). This restriction prevents the controller from producing excessive responses or going beyond the system's operating range or physical limits. The third constraint limits the maximum change in the control action using the parameter Δu_{max} . This constraint aims to prevent abrupt changes in the control action. To add flexibility to tuning, the values of M_{p_r} , U_{min} , U_{max} , and Δu_{max} are provided by the user according to the design requirements or according to the physical characteristics of the system. The remaining constraints define the search space for the controller parameters. The minimum limits for K_p , T_i , and T_d are K_{pmin} , T_{imin} , and T_{dmin} , respectively, while the

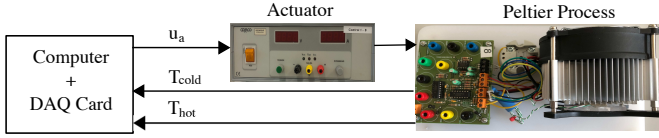


Fig. 2. Test case: Thermal process base on Peltier cell [23]

maximum limits are $K_{p_{max}}$, $T_{i_{max}}$, and $T_{d_{max}}$, respectively. For convenience, from now on, the first three constraints will be referred to as functionality constraints, while the following three will be referred to as search space constraints.

The parameters α , β , and γ weight the corresponding indices to be minimized, assigning greater or lesser importance to each. The IAE prioritizes achieving optimal performance concerning accumulated error over time. To improve accuracy, error minimization starts from k_s , an exclusion determined by the user to skip the transient period in calculating accumulated error, focusing on steady-state. Therefore, k_s should approximate the time at which the system reaches steady-state. The J_2 objective penalizes fluctuations in the control signal, which aids in regulating controller aggressiveness by minimizing cumulative variation. This, in turn, prevents excessive increases or decreases in control action. The J_3 objective penalizes transient response duration, regulating the system's speed to steady-state.

IV. OPTIMIZATION RESULTS AND DISCUSSION

A. Test case

This approach was evaluated on a lab-scale thermal process based on a Peltier cell (see Figure 2). The system input corresponds to the voltage (u_a) supplied to the Peltier cell actuator, ranging from 0 to 4.5Vdc. The temperature difference between the hot and cold surfaces (Δ_T) was regarded as the system's output. For the identification process, the input and output data were sampled at 100ms intervals. Using the Minimum Description Length (MDL) criterion, a model with one zero and four poles was selected as the best model (11). Further information on the process structure and the linear model estimation procedure can be found in [23].

$$G(s) = \frac{-41.186(s + 0.015)}{(s + 4.473)(s + 2.519)(s + 0.3406)(s + 0.014)} \quad (11)$$

B. Setup

To validate the algorithm's effectiveness, various PID controllers were tuned across two different performance scenarios. In all cases, the algorithm was configured with $N = 40$, and $t_{max} = 100$. As indicated in Subsection II-C, it was not necessary to configure additional parameters for the algorithm, as the modified version of PSO adjusts ω , c_1 , and c_2 using (4), 5, and (6), respectively, based on fixed and predefined initial and final values.

The controllers were implemented in MATLAB with an execution period of 50ms. For tuning, a set point was generated

TABLE I
PERFORMANCE OF PID CONTROLLERS OPTIMIZED WITHOUT
FUNCTIONALITY CONSTRAINTS

Code	$[\alpha \ \beta \ \gamma]$	$[K_p \ T_i \ T_d]$	$M_p(\%)$	$t_s(s)$
PID1	[1.00 0.00 0.00]	[-5.000 1.139 0.347]	45.6	1.35
PID2	[0.95 0.05 0.00]	[-1.123 1.169 0.270]	33.3	2.80
PID3	[0.55 0.05 0.40]	[-1.338 1.088 0.293]	33.3	1.75

to achieve a temperature decrease of one degree, meaning that a reference signal consisting of a step with an amplitude of -1 was established. The step time was set to 2s, and the tuning of the controllers was executed over a simulation time of 20s, i.e., $K_n = 400$. For better visualization and analysis, two scenarios were established. In one scenario, three controllers were tuned without functionality constraints but with different weights for the cost indices of the objective function. In the other scenario, another three controllers were tuned, but considering all constraints. In all optimizations, $k_s = 100$ was set for J_1 , which implies that this index was minimized from 5s. On the other hand, the search space for the controller parameters was defined as follows: $K_{p_{min}} = -5$, $T_{i_{min}} = 1e - 3$, $T_{d_{min}} = 0$, $K_{p_{max}} = 0$, $T_{i_{max}} = 5$, and $T_{d_{max}} = 1$.

C. Results and discussion

Table I presents the results of the three tuned PID controllers without functionality restrictions. It lists the assigned weights and the corresponding K_p , T_i , and T_d gains for each controller. For better reference to the controllers' performance, the overshoot (M_p) and the settling time (t_s) of the controlled variables are also shown. The performance of the three controllers can also be observed graphically in Figure 3. The PID1 controller was tuned considering only J_1 . The effect of not considering the objective J_2 ($\beta = 0$) is evident in the system's response, characterized by excessive overshoot and the amplitude reached by the control action. The PID2 controller was tuned by assigning a weight to the objective J_2 ($\beta = 0.05$). This prioritizes the cumulative variation of the control action, aiming to mitigate controller aggressiveness. It leads to a significant reduction in the maximum amplitude of the control action compared to that of the PID1 controller. This effect is also reflected in the overshoot reduction. On the other hand, The PID3 controller was tuned to prioritize minimizing settling time J_3 , reflected in its faster system response compared to the PID2 controller.

The weights for tuning the three PID controllers were chosen to demonstrate the approach's versatility and the impact of optimization objectives. Given the process's physical characteristics, none of the tuned controllers meet the required specifications, as the control action cannot exceed 4.5V. Alternatively, users can adjust the weight values to ensure the control action stays within the maximum actuator voltage. However, a more effective method would be to incorporate the functionality constraints. Table II presents the tuning of three additional controllers with all constraints applied, using the same weights as the PID3 controller: $\alpha = 0.55$, $\beta = 0.05$,

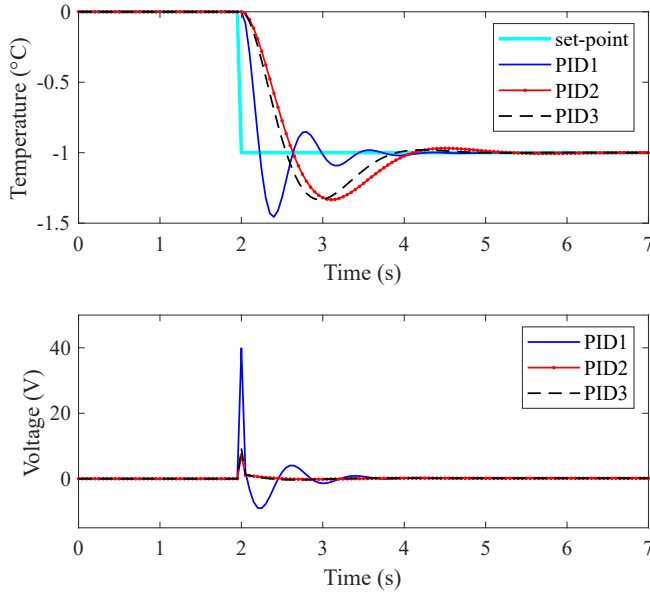


Fig. 3. PID controllers optimized without functionality constraints. Top: controlled variables. Bottom: control actions.

TABLE II

PERFORMANCE OF PID CONTROLLERS OPTIMIZED WITH FUNCTIONALITY CONSTRAINTS

Code	Δu_{max}	$[K_p \ T_i \ T_d]$	$Mp(\%)$	$t_s(s)$
PID4	0.220	$[-0.200 \ 3.105 \ 0.00]$	1.5	2.95
PID5	0.150	$[-0.147 \ 2.879 \ 0.00]$	0.46	4.40
PID6	0.120	$[-0.196 \ 3.096 \ 0.00]$	1.92	5.00

and $\gamma = 0.40$. Due to the actuator's physical limitations, the control action amplitude was restricted to $U_{min} = 0$ and $U_{max} = 4.5$, with additional constraints on the maximum variation of the control action (Δu_{max}). Figure 4 shows the performance and control actions of the three tuned controllers, highlighting how they initially react by sending maximum voltage to the Peltier cell. Due to the restriction on the maximum variation of the control action, the voltage increase is gradual. For instance, the PID4 controller's voltage rises from 0 to 0.22V between the first two samples, the maximum allowed increment. The same occurs with PID5 and PID6 controllers, considering the allowed maximums of 0.15 and 0.17, respectively.

To demonstrate the convergence effectiveness of the modified PSO algorithm, the tuning of the PID4 controller was selected and compared with three tunings performed using a standard PSO algorithm. Values for ω , c_1 , and c_2 were assigned following recommendations from the literature. Table III presents algorithm configurations and achieved objective function values, and Figure 5 compares convergence graphs. The modified PSO algorithm shows slightly better accuracy than the standard PSO across all three hyperparameter configurations, with nearly identical accuracies. Importantly, the modified PSO avoids the trial-and-error method for hyperparameter tuning and converges faster.

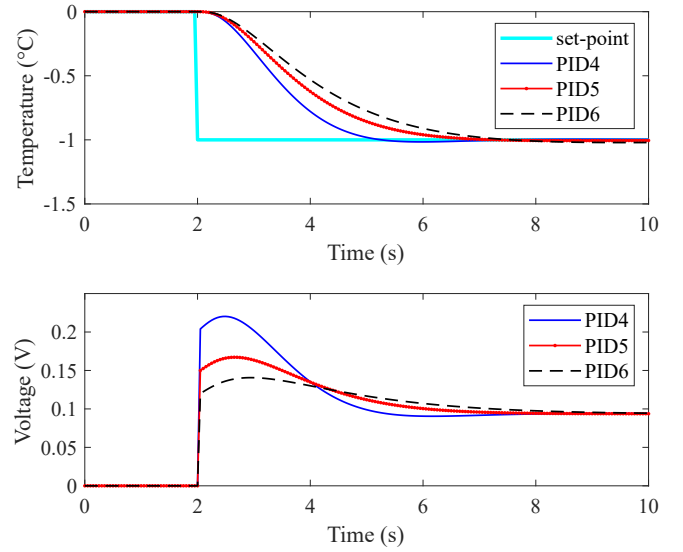


Fig. 4. PID controllers optimized with functionality constraints. Top: controlled variables. Bottom: control actions.

TABLE III

CONFIGURATION AND OBJECTIVE VALUES FOR PID4 TUNING WITH MODIFIED AND STANDARD PSO.

Tuning options	$[\omega \ c_1 \ c_2]$	J
Modified PSO	—	1.658
Standard PSO 1	$[0.729 \ 1.495 \ 1.495]$	1.659
Standard PSO 2	$[0.729 \ 2.000 \ 2.000]$	1.660
Standard PSO 3	$[1.0 \ 2.000 \ 2.000]$	1.680

V. CONCLUSIONS

Through this research, the effectiveness of a new approach for tuning PID controllers has been demonstrated. Based on a modified PSO algorithm, this approach has achieved high-precision tuning without manually adjusting the optimizer's hyper-parameters. This represents a significant advantage over conventional methods, as it simplifies the application of the algorithm by eliminating the need for user intervention during initial calibration. Additionally, unlike traditional tuning methods that focus on minimizing a single type of error, the proposed approach provides a more comprehensive and effective balance between various objectives, which are simultaneously minimized to achieve good system performance while considering the behavior of the control action. Furthermore, incorporating constraints within the optimization problem has allowed for precise management of certain parameters of both the system response and the control action. This ensures that the tuned controllers not only improve system performance but also comply with the physical limitations of the process, such as the amplitude and maximum variation of the control signal.

ACKNOWLEDGMENT

The authors thank the Salesian Polytechnic University of Ecuador for supporting this work through the Research Group on Interaction, Robotics, and Automation.

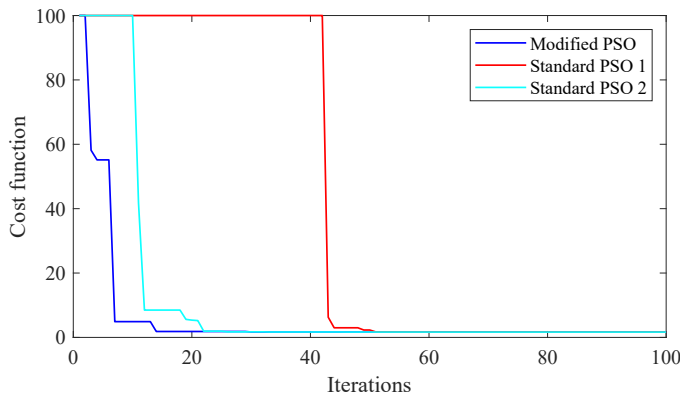


Fig. 5. Conv. graf

REFERENCES

- [1] K. H. Ang, G. Chong, and Y. Li, "Pid control system analysis, design, and technology," *IEEE transactions on control systems technology*, vol. 13, no. 4, pp. 559–576, 2005.
- [2] K. J. Åström and T. Hägglund, "The future of pid control," *Control engineering practice*, vol. 9, no. 11, pp. 1163–1175, 2001.
- [3] R. P. Borase, D. Maghade, S. Sondkar, and S. Pawar, "A review of pid control, tuning methods and applications," *International Journal of Dynamics and Control*, vol. 9, pp. 818–827, 2021.
- [4] K. J. Åström and T. Hägglund, "Revisiting the ziegler–nichols step response method for pid control," *Journal of process control*, vol. 14, no. 6, pp. 635–650, 2004.
- [5] C. C. Hang, K. J. Åström, and W. K. Ho, "Refinements of the ziegler–nichols tuning formula," in *IEE Proceedings D (Control Theory and Applications)*, IET, vol. 138, 1991, pp. 111–118.
- [6] E. Joseph and O. Olaiya, "Cohen-coon pid tuning method; a better option to ziegler nichols-pid tuning method," *Engineering Research*, vol. 2, no. 11, pp. 141–145, 2017.
- [7] D. E. Rivera, M. Morari, and S. Skogestad, "Internal model control: Pid controller design," *Industrial & engineering chemistry process design and development*, vol. 25, no. 1, pp. 252–265, 1986.
- [8] S. Skogestad and C. Grimholt, "The simc method for smooth pid controller tuning," *PID control in the third millennium: Lessons learned and new approaches*, pp. 147–175, 2012.
- [9] N. Thomas and D. P. Poongodi, "Position control of dc motor using genetic algorithm based pid controller," in *Proceedings of the world congress on engineering*, Citeseer, vol. 2, 2009, pp. 1–3.
- [10] J.-G. Juang, M.-T. Huang, and W.-K. Liu, "Pid control using presearched genetic algorithms for a mimo system," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 5, pp. 716–727, 2008.
- [11] M. S. Saad, H. Jamaluddin, and I. Z. M. Darus, "Implementation of pid controller tuning using differential evolution and genetic algorithms," *International Journal of Innovative Computing, Information and Control*, vol. 8, no. 11, pp. 7761–7779, 2012.
- [12] Z. Bingul, "A new pid tuning technique using differential evolution for unstable and integrating processes with time delay," in *International Conference on Neural Information Processing*, Springer, 2004, pp. 254–260.
- [13] M. I. Solihin, L. F. Tack, and M. L. Kean, "Tuning of pid controller using particle swarm optimization (pso)," *International Journal on Advanced Science, Engineering and Information Technology*, vol. 1, no. 4, pp. 458–461, 2011.
- [14] S. GirirajKumar, D. Jayaraj, and A. R. Kishan, "Pso based tuning of a pid controller for a high performance drilling machine," *International Journal of Computer Applications*, vol. 1, no. 19, pp. 12–18, 2010.
- [15] G. Štimac, S. Braut, and R. Žigulić, "Comparative analysis of pso algorithms for pid controller tuning," *Chinese Journal of Mechanical Engineering*, vol. 27, pp. 928–936, 2014.
- [16] M. Omar, M. Soliman, A. A. Ghany, and F. Bendary, "Optimal tuning of pid controllers for hydrothermal load frequency control using ant colony optimization," *International journal on electrical engineering and informatics*, vol. 5, no. 3, pp. 348–360, 2013.
- [17] L. Fan and E. M. Joo, "Design for auto-tuning pid controller based on genetic algorithms," in *2009 4th IEEE conference on industrial electronics and applications*, IEEE, 2009, pp. 1924–1928.
- [18] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, 1995, 1942–1948 vol.4. DOI: 10.1109/ICNN.1995.488968.
- [19] M. Jain, V. Saihpal, N. Singh, and S. B. Singh, "An overview of variants and advancements of pso algorithm," *Applied Sciences*, vol. 12, no. 17, p. 8392, 2022.
- [20] F. Marini and B. Walczak, "Particle swarm optimization (pso). a tutorial," *Chemometrics and Intelligent Laboratory Systems*, vol. 149, pp. 153–165, 2015.
- [21] R. C. Eberhart and Y. Shi, "Comparing inertia weights and constriction factors in particle swarm optimization," in *Proceedings of the 2000 congress on evolutionary computation. CEC00 (Cat. No. 00TH8512)*, IEEE, vol. 1, 2000, pp. 84–88.
- [22] A. Ratnaweera, S. Halgamuge, and H. Watson, "Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients," *IEEE Transactions on Evolutionary Computation*, vol. 8, no. 3, pp. 240–255, 2004. DOI: 10.1109/TEVC.2004.826071.
- [23] J. Zambrano, J. Sanchis, J. Herrero, and M. Martínez, "A unified approach for the identification of wiener, hammerstein, and wiener–hammerstein models by using wh-ea and multistep signals," *Complexity*, vol. 2020, pp. 1–23, 2020.